Application of LLM on Games

1. Abstract

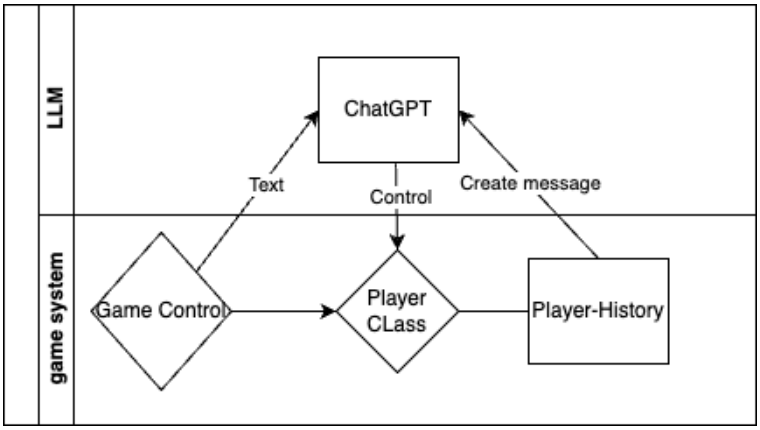
In the previous stages, the LLM was tested with the Chinese card game "Landlord." Despite the earlier findings with ChatGLM, several improvements have occurred since the last report. The game system has been upgraded with better prompt engineering feasibility and also implemented with OpenAI API to have a wider selection of LLM models. This report introduced a new method to evaluate the LLM in understanding gaming to compare the performance between different models. And will use the method to evaluate different prompt engineering approaches. Among several popular prompts that potentially increase model performance, this report discusses several prompts that would influence the performance of the model in card games. In the end, this report will discuss further improvements for gaming LLM in the next stage.

2. Introduction of the game

Landlord is a Chinese poker game played with one pack of cards. There will be three players by bidding; one of them is the landlord, and the other two of them are peasants. The objective of the peasants is to beat the landlord by casting cards faster than the landlord. Once one of the peasants has no cards left, the peasants will win the game, and vice versa. There are several ways to cast cards, from single cards to pairs straight and so on. Casting multiple cards in turn (i.e., casting straight instead of singles) will accelerate the gameplay and give more chances for a player to win [1, 2].

3. Modification of the program structure

The program mainly contains two parts. The game control program will take inputs from the human player or the LLM player, with prompts to guide the player. And processing input from the player to take game actions (e.g., casting cards). In the previous report, the game system is implemented with message passing from a locally running LLM program with the ChatGLM-6B model. Now, it has changed with interaction with OpenAI API. With the latest ChatGPT API, there will be a wider choice of models and an improvement in the performance of the game system. The code has been uploaded to Github. More details will be shown on the repository.



To ensure the consistency of LLM reaction, A new data member ‘history’ will be added to the player class. With the history of the game being shown to the LLM, it will enhance the understanding of the LLM.

The following information about the game will be shown to the LLM before every turn:

- The series on the table
- The cards on their hands
- The system announcements (who is the landlord, and the landlord’s extra cards)
- All the card-casting history of all players

- System information(invalid casting)

4. Performance evaluation

There are several criteria for evaluating the performance of LLMs, but they do not apply to our attempt to let LLMs play card games [3]. We introduce two criteria to evaluate the performance of LLMs. The first criterion is the error rate, which can evaluate the understanding of the game rules since violating the game rules will cause the game control program to penalize. When LLM is making an incorrect reaction, the game control program will accumulate an error count to calculate the error rate of the LLM performance. The rate has two calculations used in this experiment, and more information will be in the future steps chapter.

$$error_rate = \frac{number\ of\ correct\ output}{number\ of\ output}$$

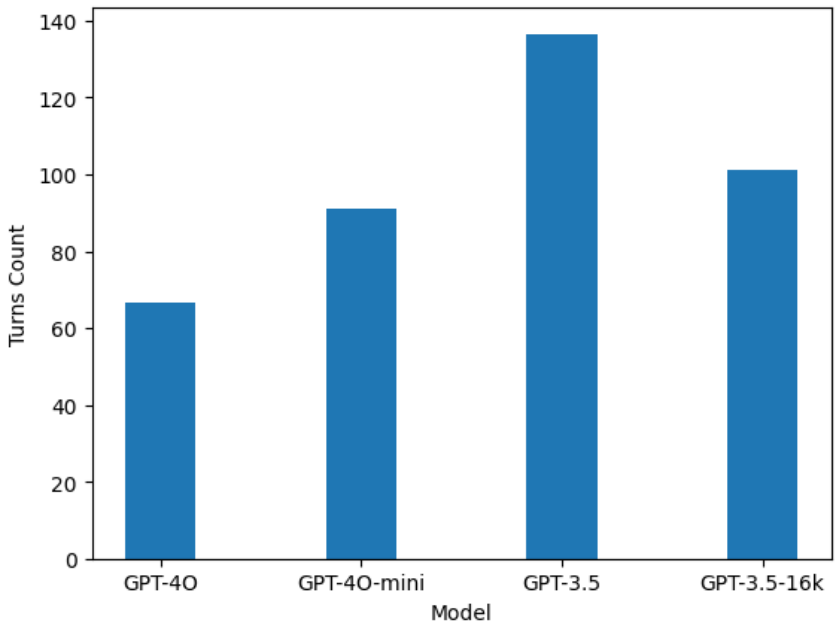
$$error_rate = \frac{number\ of\ correct\ output}{number\ of\ turns}$$

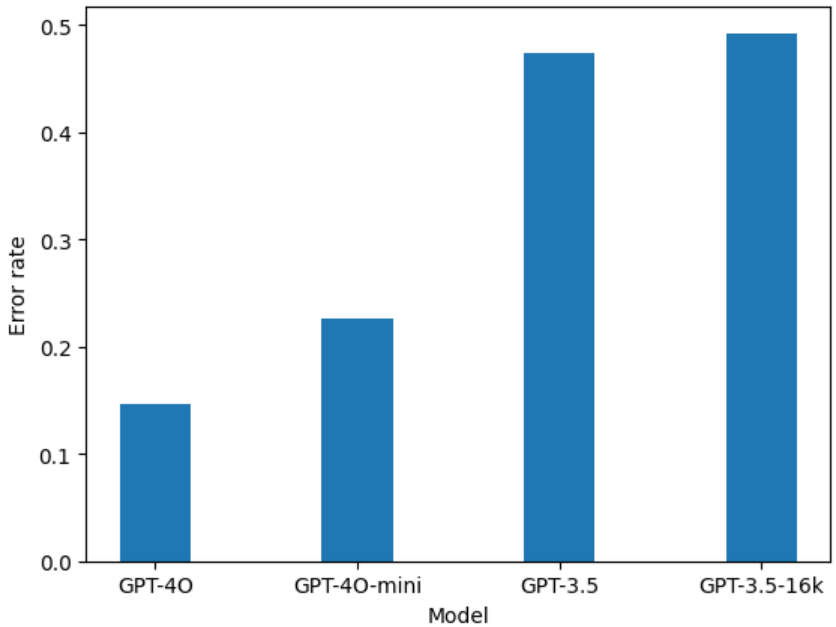
The second criterion is the number of turns to end the game. This criterion can evaluate the LLM performance by the ability to win the game. With a better understanding of the games, the LLM player can make moves to cast their cards quickly. For example, if the players are casting pairs instead of singles, if applicable, the game will be shorter.

5. Compare between models

This experiment will first compare the performance of LLM gaming with different models from OpenAI. Before starting our comparison, we found that GPT-3.5 cannot cast cards correctly. The lack of game knowledge makes it cast cards randomly. With a slight

modification of the code, the infinite loop is prevented. Although this won't improve the performance of GPT-3.5, it is now comparable with other models.





From the table above, we can see that GPT-4o has the best performance in our evaluation criteria. The error rate of GPT-4o is less than 0.2. While GPT-4o-mini also has a great performance. However, the other models, GPT-3.5 and GPT-3.5-16k, are not performing well from the observation of the experiment. Before we add the infinite loop prevention algorithm

(which will end a player's turn if the player keeps casting invalid cards), the error rate of GPT-3.5 is near 99% (most of the responses from the model are not valid). It wastes a lot of resources since every attempt of the cast will take LLM to run an inference. The results roughly suggest that the performance will be improved as the size of LLM increases [4]. The largest LLM GPT-4o performs best with the least error rate and turn count. We will use GPT-4o as the base model to perform prompt engineering.

6. Modification of Prompts

Prompt Engineering is the discipline of making precise prompts to let the LLM have an accurate reaction [5, 6]. In this experiment, prompt engineering is used to let LLM have accurate casting of cards based on the game's rules. And also commands to adjust the performance of the LLM player. Two parts of the code involve Prompt Engineering. The first one is the general command to explain the game information to the LLM. The following paragraphs are the initial prompts entered into the LLM.

*[previous player name] cast cards [series]**

*You are [player name], your cards are [cards], the previous player casts [series], and enter PASS to skip this round. The houses are ♠, ♥, ♣, ♦, enter Joker to cast jocker. Please answer the cards to cast, for example: ♥3**

*You are [player name], your cards are [cards], the previous players chose to skip, and you can cast whatever you want but cannot skip this round. The houses are ♠, ♥, ♣, ♦, enter Joker to cast jocker. Please answer the cards to cast, for example: ♥3**

*The cards you cast are not valid. Please cast them again.**

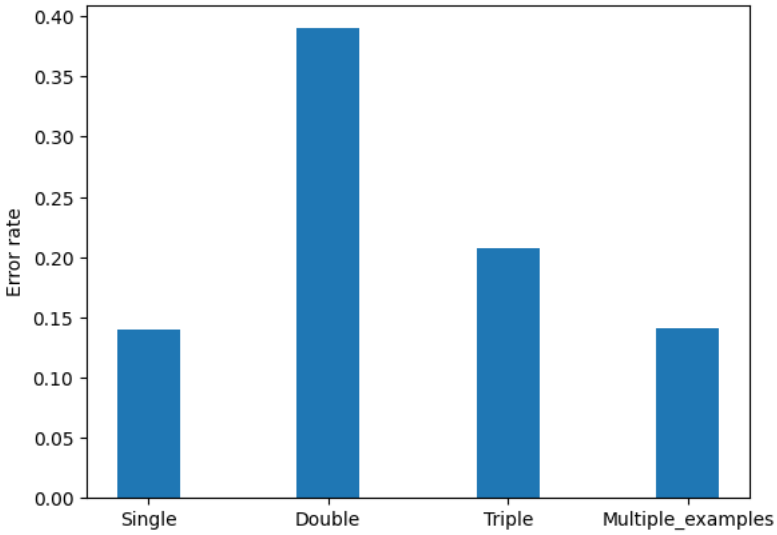
These commands have been tested to guide the player to have an accurate understanding of the situation of the game and can also guide human players to play the game. But in the results, the reaction of the player is mainly single cards, which is the same as the prompt ♥*3*. Keep casting single cards will extend the time of the game. The following examples are tested.

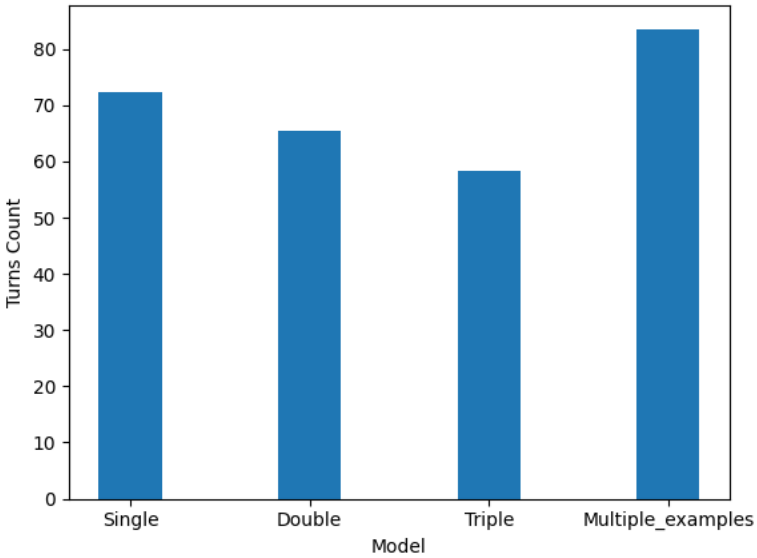
[Single]♠*3*

[Double]♠*3*♥*3*

[Triple]♠*3*♥*3*♣*3*

[Multiple examples]♠*3 or* ♠*3*♥*3 or* ♠*3*♥*3*♣*3*





By modifying the prompt's example, it can show an influence made by the prompt [7]. From single to triples, the number of turns decreases. This is explainable since casting doubles and triples can accelerate the gameplay, and the number of turns indeed decreases. Although the error rate can easily be influenced by outliers, the data suggest that using doubles and three triples will have an increase in error rate. Checking the raw output data shows that the LLM player tried to cast doubles and triples when the previous player was casting singles (which violates the game rule). The last prompt in the experiment, which is the one that gives multiple examples of the LLM, has similar results to the prompt with a single. This might be due to the single example being placed in the first place.

At the start of the game, the system can give a role to the LLM. By a prompt telling the model its role in playing the game, there are differences in the performance of the model. The following prompts are tested.

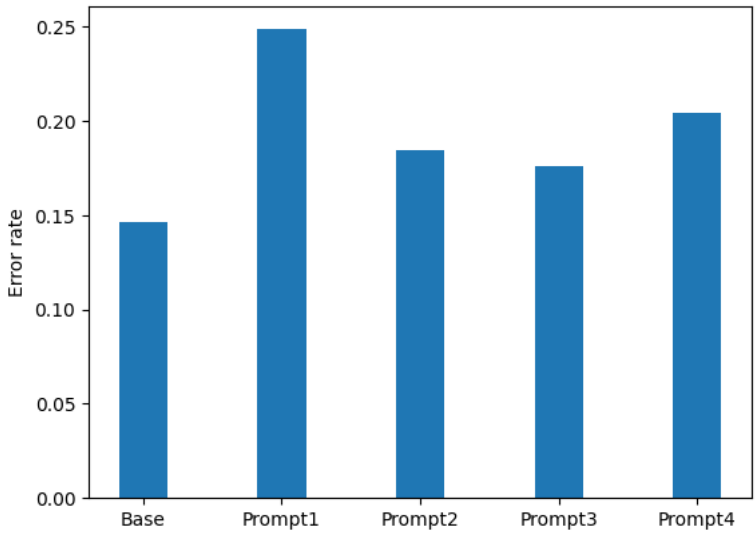
[Base]*You are an artificial intelligence that plays Landlords**

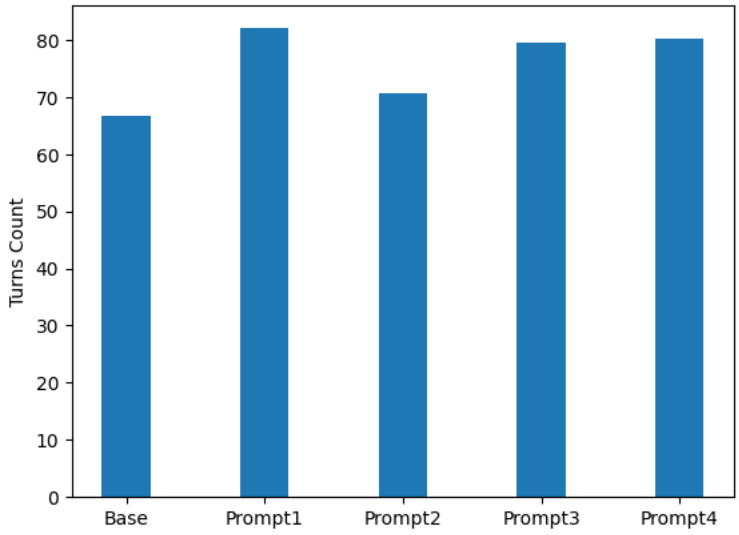
[Prompt1] *You are the champion of Landlords**

[Prompt2] *You like to play pairs, three of a kind, and straights the most**

[Prompt3] *I want you to become a Landlord robot, the rules are as follows: [link of game rules] **

[Prompt4] *I want you to become a Landlord robot, you are very familiar with the rules and winning methods**





There are some modifications to the execution codes between the current and the previous experiments. Please ignore the results difference between experiments.

We can tell that the base prompt has the best performance among others, while there are minor differences between others. The first prompt has the worst performance in terms of both error rate and turn taken. This can be related to the unclear role given to the LLM.

By giving a detailed explanation of the game from a website, prompt3 has a lower error rate than prompt4 but still outperforms the base prompt. This suggests that a clear and straight prompt given to the LLM will have a better performance than others.

*Translated from Chinese. The original version is in the repository.

7. Further steps

In the step of training an LLM, there is a step called Reinforced Learning with Human Feedback (RLHF), by ranking data by humans as rewards to train the LLM, which gives the LLM game playing a new sight of training. By using the current evaluation (error rate and turns count) as a reward, it is possible to train the LLM to play the game with better performance in a similar way to RLHF. Also, since the Landlord is a Player vs Player game, it is possible to use the winning rate as a criterion to train the model [8]. In the next step of this project, this learning attempt will be tried on open-source models. The model can be trained with awards given by error rate to increase stability, by number of turns to increase the use of pairs and triples, and also by award of winning to increase the winning rate. Supervised Fine-tuning(SFT) is also a way to train the model. Giving questions and answers to concepts from the Landlord game might have a better performance than giving plain prompts about the rules of the game. SFT can also be an approach for game-playing LLM [9]. This approach can also be tested in future steps.

8. Conclusion

To this end, we found that the performance of LLM can be improved by the model's size and better prompts. With two special criteria to measure the performance of LLM in different aspects, experiments on prompts are conducted. While some of the prompts are found to be underperformance, there is not the best solution for prompts. There roughly exists a trade-off between error rate and the number of turns. Although error won't affect the game performance of the LLM, it will waste computation resources. The trade-off can also be regarded as between cost (error rate) and performance (count of turns).

Reference

[1] Wikipedia contributors, “Dou dizhu,” *Wikipedia*, Dec. 07, 2024. https://en.wikipedia.org/wiki/Dou_dizhu

[2] D. Zha *et al.*, “DouZero: Mastering DouDizhu with Self-Play Deep Reinforcement Learning,” *arXiv.org*, Jun. 11, 2021. https://arxiv.org/abs/2106.06135

[3] S. Karzhev, “LLM Evaluation: metrics, methodologies, best practices,” Aug. 06, 2024. https://www.datacamp.com/blog/llm-evaluation

[4] Amity Solutions Co., Ltd., “GPT-4o vs GPT-4o Mini: Choosing the Right AI Model,” *Amity Solutions Co., Ltd.*, Aug. 02, 2024. https://www.amitysolutions.com/blog/chatgpt-35-vs-chatgpt-4

[5] “Basics of prompting – Nextra,” Nov. 18, 2024. https://www.promptingguide.ai/introduction/basics

[6] “Awesome ChatGPT prompts,” *Awesome ChatGPT Prompts*. https://prompts.chat/

[7] M. V, “Prompt engineering for game development,” *Analytics Vidhya*, Jun. 24, 2024. https://www.analyticsvidhya.com/blog/2024/06/prompt-engineering-for-game-development/

[8]D. M. Ziegler *et al.*, “Fine-Tuning Language Models from Human Preferences,” *arXiv.org*, Sep. 18, 2019. https://arxiv.org/abs/1909.08593

[9] C. R. Wolfe PhD, “Understanding and using Supervised Fine-Tuning (SFT) for language models,” *Deep (Learning) Focus*, Sep. 11, 2023. https://cameronrwolfe.substack.com/p/understanding-and-using-supervised